ASP.NET Core Error Handling និង Validation

ឯកសារនេះផ្តល់នូវការណែនាំយ៉ាងលម្អិតអំពីរបៀបគ្រប់គ្រងកំហុស (Error Handling) ការ Validate ទិន្នន័យ (Model Validation) ការតម្លើង Health Checks និងការធ្វើ Monitoring and Observability ក្នុងកម្មវិធី ASP.NET Core Web API។ ឯកសារនេះត្រូវបានរចនាឡើងសម្រាប់អ្នកអភិវឌ្ឍន៍ដែលមានបទពិសោធន៍មធ្យម ដើម្បីឈានទៅកម្រិតខ្ពស់ និងសម្រេចបាននូវប្រព័ន្ធដែលមានស្ថិរភាព។

ERROR HANDLING

១. អំពីការគ្រប់គ្រងកំហុស (Error Handling)

នៅពេលមាន **Exception** កើតឡើងក្នុងកម្មវិធី **ASP.NET Core Web API** កម្មវិធីនឹងបោះ (throw) exception មួយចេញ។ បើ exception នេះមិនត្រូវបានចាប់យក (handle) ទេ កម្មវិធីអាចបញ្ឈប់ដំណើរការ (crash) ហើយបង្កើតជា **HTTP 500 Internal Server Error**។

Development Environment

ការបង្ហាញ Stack Trace ទៅ Client គឺអាចទទួលយកបាន ព្រោះអ្នកអភិវឌ្ឍន៍ត្រូវការព័ត៌មានលម្អិតដើម្បី Debug។

Production Environment

មិនគួរបង្ហាញ Stack Trace ទៅ Client ឡើយ ព្រោះវាអាចបង្ហាញព័ត៌មានសំខាន់អំពីប្រព័ន្ធ ហើយបង្កហានិភ័យផ្នែកសុវត្ថិភាព។

ក្នុងករណីដែល exception មិនត្រូវបានចាប់យក **Response Body** នឹងមានព័ត៌មានលម្អិតអំពីកំហុស រួមទាំង **Stack Trace** នៃ exception ផងដែរ។ ដូច្នេះ ការរៀបចំប្រព័ន្ធ Error Handling ដ៏មានប្រសិទ្ធភាពគឺចាំបាច់ណាស់ ដើម្បីការពារប្រព័ន្ធ និងបង្ហាញសារកំហុសសមស្រប។

មូលដ្ឋានគ្រឹះ

១.១. តើ Exception ជាអ្វី?

Exception (ករណីកំហុសពេលដំណើរការ) គឺជាព្រឹត្តិការណ៍កំហុស (error event) ដែលកើតឡើងនៅពេលកម្មវិធីកំពុងដំណើរការ (runtime) ហើយរំខានដល់លំហូរធម្មតានៃកូដ។

Runtime Error

Exception កើតឡើងនៅពេល runtime មិនមែននៅពេល compile time ទេ។ ឧទាហរណ៍ ការចូលប្រើ Array ដែលហួសសន្ទស្សន៍ (Index Out of Range) ឬ ការបែងចែកលេខដោយ Zero។

Interrupts Flow

Exception ធ្វើឲ្យលំហូរធម្មតានៃការដំណើរការកូដត្រូវបានរំខាន ហើយ control ត្រូវបានផ្ទេរទៅ exception handler ដែលនៅជិតបំផុត។

Unhandled Exception

ប្រសិនបើ Exception មិនត្រូវបាន Handle វានឹងធ្វើឲ្យ Application Crash ហើយ Server ត្រឡប់ HTTP 500 Internal Server Error ទៅ Client។

ហេតុផល

១.២. ហេតុអ្វីត្រូវប្រើ Exception?

Exception ត្រូវបានប្រើ ដើម្បីគ្រប់គ្រងករណី "មិនធម្មតា" (unexpected situations) នៅពេលកម្មវិធីកំពុងដំណើរការ (runtime) ហើយធ្វើឲ្យប្រព័ន្ធមានសុវត្ថិភាព និងអាចគ្រប់គ្រងបាន។

1 គ្រប់គ្រង Unexpected Situations

ជួយគ្រប់គ្រងស្ថានភាពដែលមិននឹកស្មានដល់ ដូចជា Database Connection Failed, File Not Found, Network Timeout ជាដើម ដោយមិនឲ្យ Application Crash។

2 ញែកកូដធម្មតា ចេញពីកូដ Error

ការប្រើ try-catch ជួយញែក business logic ចេញពីការគ្រប់គ្រង error ដែលធ្វើឲ្យកូដកាន់តែស្អាត និងអានកាន់តែងាយ។

3 ផ្ញើ Meaningful Error Messages

ជំនួសឲ្យការបង្ហាញ Error ពួក Technical ដែលអ្នកប្រើប្រាស់មិនយល់ Exception Handling ជួយបម្លែងបន្ថែម Error ទៅជា User-Friendly Messages។

4 Logging & Debugging

Exception ក៏ជាឱកាសសម្រាប់ Log ព័ត៌មានលម្អិតអំពី Error ដើម្បីឲ្យ Developer អាច Debug ចំណុចបញ្ហាបានយ៉ាងរហ័ស។

ប្រភេទ

១.៣. ប្រភេទនៃ Exception

ក្នុង .NET/C# Exception ត្រូវបានបែងចែកជាប្រភេទផ្សេងៗ ដោយ Exception ទាំងអស់ Inherit មកពី Base Class **System.Exception**។ ការយល់ដឹងពីប្រភេទ Exception នីមួយៗ នឹងជួយឲ្យអ្នកអភិវឌ្ឍន៍ Handle Exception បានត្រឹមត្រូវ និងមានប្រសិទ្ធភាព។

ប្រភេទ **Exception**	**ការពិពណ៌នា**	**ឧទាហរណ៍**
NullReferenceException	ចូលប្រើ Object ដែល null	string s = null; s.Length;
ArgumentException	Argument មិនត្រឹមត្រូវ	Method ទទួលតម្លៃដែលមិនត្រឹមត្រូវ
InvalidOperationException	Operation មិនអាចធ្វើបានក្នុង state នេះ	Stack ទទេ ប៉ុន្តែចង់ Pop
IOException	File ឬ Network operation fail	File Not Found, Disk Full
UnauthorizedAccessException	គ្មានសិទ្ធិ access	Read-only file, Permission denied
Custom Exception	Exception ដែលអ្នកបង្កើតខ្លួនឯង	BusinessException, NotFoundException

បច្ចេកទេស

១.៣.១. ការប្រើប្រាស់ try-catch

គ្រោងឆ្អឹង **try-catch** គឺជារបៀបមូលដ្ឋានបំផុតក្នុងការ Handle Exception ក្នុង C#។ try block ដំណើរការកូដដែលអាចបោះ exception ហើយ catch block ចាប់ exception ហើយ Handle វា។

Try Block
ដំណើរការកូដដែលអាចបង្កើត exception។

Finally Block
ធ្វើសកម្មភាពសម្អាតនៅជានិច្ច។

Catch Block
ចាប់ exception និងដោះស្រាយ កំហុស។

finally block គឺតែងដំណើរការជានិច្ច មិនថា exception កើតឡើង ឬអត់ ដូច្នេះវាត្រូវបានប្រើសម្រាប់ cleanup operations ដូចជា បិទ database connection, release resources ជាដើម។ ការ stack-ធ្វើ catch blocks ច្រើន (multiple catch) ជួយចាប់ exception ប្រភេទផ្សេងៗ ហើយ Handle ពួកវាដោយឡែកពីគ្នា។

> ចំណាំ៖ Catch ប្រភេទ Exception ដែលជាក់លាក់ (specific) ជាមុនជានិច្ច ហើយ catch Exception ទូទៅ (base Exception) ជាចុងក្រោយ ដើម្បីប្រើ catch blocks ឲ្យបានត្រឹមត្រូវ។

GLOBAL

១.៣.២. Global Exception Handling ជាអ្វី?

Global Exception Handling គឺជាយន្តការដែលចាប់ Exception ទាំងអស់ ដែលមិនត្រូវបាន Handle នៅកន្លែងណាមួយក្នុង Application ហើយ Handle ពួកវានៅកន្លែងតែមួយ (centralized location)។

Centralized Error Management

ជំនួសឲ្យការ Handle Exception នៅគ្រប់ endpoint ម្តងម្នាក់ Global Exception Handler ធ្វើការ Handle Error ក្នុងកន្លែងតែមួយ ដែលកាត់បន្ថយ code duplication។

Consistent Error Responses

ធ្វើឲ្យ Error Response ទាំងអស់ត្រូវបានបង្ហាញក្នុង format តែមួយ (ដូចជា ProblemDetails) ដែលធ្វើឲ្យ Client ដឹងពីរបៀប Handle Error ងាយស្រួល។

Prevent Information Leakage

ការបង្ហាញ Stack Trace ទៅ Client ក្នុង Production Environment គឺហានិភ័យផ្នែកសុវត្ថិភាព។ Global Handler ការពារ sensitive information ពីការ leak ចេញ។

ហេតុផល

ក. ហេតុអ្វីត្រូវប្រើ Global Handling?

ប្រសិនបើ Application ធំ ហើយមាន Endpoint ច្រើន ការ Handle Exception នៅគ្រប់ Endpoint នឹងនាំឲ្យ Code ម្នាក់ Duplicate ច្រើន។ Global Exception Handling ដោះស្រាយបញ្ហានេះ ដោយ Handle Exception ក្នុងកន្លែងតែមួយ (Middleware ឬ Filter)។

ចៀសវាង Code Duplication

Handle Error ម្តង, apply ទៅ Application ទាំងមូល។

Consistent Responses

Response format ដូចគ្នាក្នុង Endpoint ទាំងអស់ ជួយ Client ងាយ Parse Error។

Security

លាក់ Technical Detail (Stack Trace) ពី Client ក្នុង Production Environment។

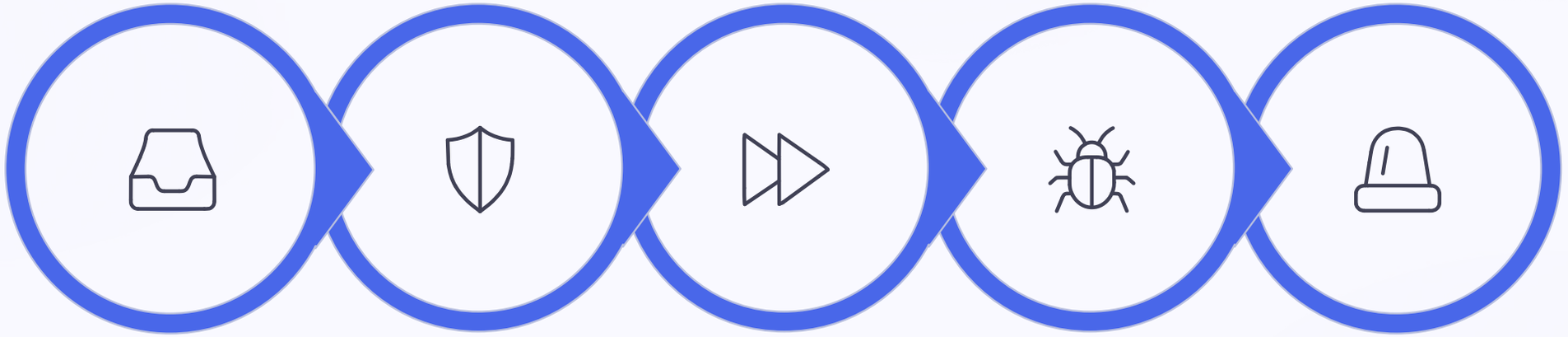
Logging

Log Exception ទាំងអស់នៅ centralized location ដើម្បី monitor Application health។

MIDDLEWARE

ខ. ការគ្រប់គ្រងកំហុសជាមួយ Middleware

ASP.NET Core ផ្តល់នូវវិធីច្រើនដើម្បីធ្វើ Global Exception Handling ក្នុងចំណោមនោះ **Exception Handling Middleware** គឺជាវិធីដ៏ពេញនិយម និងមានអានុភាព។ Middleware នឹងអន្តរាគមន៍ (intercept) Request/Response pipeline ហើយចាប់ Exception ណាមួយដែលកើតឡើង។



ការប្រើ **UseExceptionHandler middleware** ឬបង្កើត Custom Exception Middleware ផ្ទាល់ខ្លួន អ្នកអភិវឌ្ឍន៍អាចចាប់ Exception ទាំងអស់ Format Error Response ឲ្យ consistent ហើយ Log ព័ត៌មាន Error ចូល logging system ។ ក្នុង ASP.NET Core .NET 8+ មាន **IExceptionHandler** interface ថ្មីដែលធ្វើឲ្យការ Handle Exception កាន់តែស្អាត។

PROBLEMDETAILS

គ. ProblemDetails ជាអ្វី?

ProblemDetails គឺជា Standard Format សម្រាប់ Error Responses ក្នុង HTTP API ដែលត្រូវបានកំណត់ដោយ **RFC 7807**។ ASP.NET Core បានរួមបញ្ចូល ProblemDetails នៅក្នុង Framework ដោយ Default ដើម្បីធ្វើឲ្យ Error Responses ត្រូវបានបង្ហាញក្នុង Standard Format។

> RFC 7807 (Problem Details for HTTP APIs) – ជា Standard ដែលកំណត់ Format ស្តង់ដារសម្រាប់ Error Responses ក្នុង HTTP APIs ដែល API ផ្សេងៗ ដូចជា Microsoft, Google, Stripe សុទ្ធតែប្រើ Standard នេះ។

ការប្រើ ProblemDetails ជួយធ្វើឲ្យ Error Responses ក្នុង API របស់អ្នកស្ថិតក្នុង Format ស្តង់ដារ ហើយ Client (Frontend App, Mobile App, Third-party Service) អាចដោះស្រាយ Error បានងាយ ព្រោះ Format គឺ Predictable និង Consistent។

STRUCTURE

គ.១. ProblemDetails Structure

ProblemDetails Object មាន Properties ស្តង់ដារដែលកំណត់ដោយ RFC 7807 ។ Properties ទាំងនេះអនុញ្ញាតឲ្យ Client ដឹងពីប្រភេទ Error, HTTP Status Code, Message, និង Link ទៅ Documentation។

Property	ប្រភេទ	ការពណ៌នា
type	string (URI)	URI Reference ដែលបញ្ជាក់ប្រភេទ Error (អាច Link ទៅ Documentation)
title	string	ចំណងជើងខ្លីអំពី Error (Human-readable)
status	integer	HTTP Status Code (ឧ. 400, 404, 500)
detail	string	ការពណ៌នាលម្អិតអំពី Error ជាក់លាក់
instance	string (URI)	URI Reference ដែលបញ្ជាក់ instance ជាក់លាក់ of Error
extensions	dictionary	Properties បន្ថែម Custom ដូចជា errors, traceId ជាដើម

អត្ថប្រយោជន៍

គ.២. អត្ថប្រយោជន៍ ProblemDetails

Standard Format

ប្រើ RFC 7807 Standard ដែលជា Industry Best Practice ធ្វើឲ្យ API Professional ហើយ Developer ផ្សេងៗ យល់ Format Error ភ្លាម។

Built-in ASP.NET Core

ASP.NET Core .NET 7+ បាន Support ProblemDetails ដោយ Default។ គ្រាន់តែ Register Service ហើយ Framework ធ្វើ Format Error ឲ្យអ្នក ដោយ Automatic។

Extensible

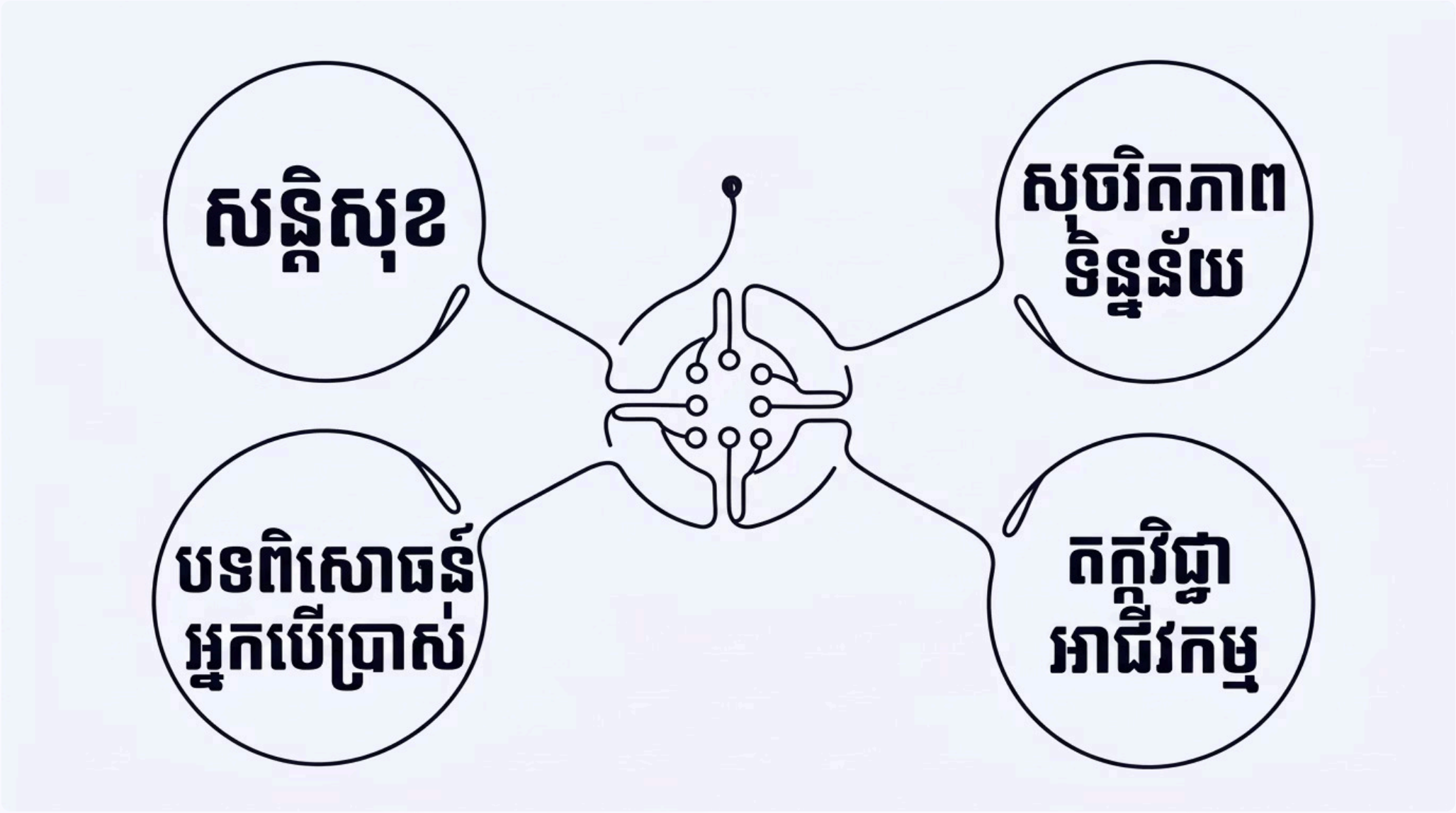
អ្នក អាច Extend ProblemDetails ដើម្បី Add Custom Properties ដូចជា errors (Validation errors), traceId ឬ correlationId ជាដើម ដែលជួយ Debug ច្រើន។

ក្នុង ASP.NET Core .NET 7 ឡើងទៅ ការ Register **builder.Services.AddProblemDetails()** នឹងធ្វើឲ្យ Framework ប្រើ ProblemDetails format ដោយ Automatic សម្រាប់ HTTP Error Responses ទាំងអស់។

MODEL VALIDATION

២. អំពី Model Validation

Model Validation គឺជាដំណើរការពិនិត្យ (verify) ទិន្នន័យ (data) ដែល Client ផ្ញើមក (Request Body, Query String, Route Parameters) ថាតើ ទិន្នន័យនោះ ត្រឹមត្រូវ (valid) ស្របតាមច្បាប់ (rules) ដែលយើងបានកំណត់ ឬ អត់ ។



ការ Validate Model ជួយការពារ Application ពីការទទួល ទិន្នន័យមិនត្រឹមត្រូវ (invalid data) ដែលអាចបណ្តាលឲ្យមាន Security Vulnerabilities, Data Corruption ឬ Business Logic Errors។ ការ Validate ត្រឹមត្រូវ ក៏ជួយ User Experience ផងដែរ ព្រោះ Error Messages ច្បាស់ លាស់ ជួយ User ដឹងថា ពួកគេ Submit Data ខុស ។

ការពន្យល់

២.១. តើ Model Validation ជាអ្វី?

Model Validation ក្នុង ASP.NET Core គឺជាដំណើរការ Automatic ដែល Framework ប្រើដើម្បី Validate Data ដែល Client ផ្ញើមក ហើយ Bind ទៅ Action Parameters ឬ Models។ ដំណើរការ Validation នឹងជោគជ័យ ឬ Fail ដោយ Automatic ហើយ Framework ត្រឡប់ 400 Bad Request Response ប្រសិនបើ Validation Fail។

1

Define Rules

កំណត់ Validation Rules ដោយប្រើ Data Annotations ឬ Fluent Validation ក្នុង Model Class។

2

Bind Data

ASP.NET Core Bind Data ពី Request (Body, Query, Route) ទៅ Model Object ដោយ Automatic។

3

Validate

Framework Validate Model Object ប្រឆាំងនឹង Rules ហើយ Populate ModelState ។

4

Return Result

ប្រសិនបើ Valid, Controller Action ដំណើរការ។ ប្រសិនបើ Invalid, Framework ត្រឡប់ 400 Bad Request ដោយ Automatic។

ដំណើរការ

២.២. តើ Model Validation ដំណើរយ៉ាងដូចម្តេច?

ក្នុង ASP.NET Core Web API ដំណើរការ Model Validation ត្រូវបានគ្រប់គ្រងដោយ **Model Binding** និង **Model State**។ Framework ប្រើ **[ApiController]** Attribute ដើម្បី Enable Automatic Model Validation Response។

ModelState.IsValid គឺជា Property ដែលរក្សាស្ថានភាព Validation – **true** ប្រសិនបើ Model ត្រឹមត្រូវ ហើយ **false** ប្រសិនបើ Model មានបញ្ហា។ ក្នុង API Controllers ដែលប្រើ **[ApiController]** Attribute ការ Check ModelState.IsValid ត្រូវបានធ្វើដោយ Automatic – ប្រសិនបើ Validation Fail, Framework ត្រឡប់ 400 Bad Request Response ភ្លាម ដោយ Automatic ហើយ Code ក្នុង Action Method នឹងមិនដំណើរការ។

កន្លែង

២.៣. តើ Model Validation ធ្វើនៅឯណាខ្លះ?

Validation អាចធ្វើបានក្នុងកន្លែងច្រើននៅក្នុង Application Architecture ហើយ Best Practice គឺ Validate ទិន្នន័យ ក្នុងស្រទាប់ (layer) ផ្សេងៗ ដើម្បីធានាថា ទិន្នន័យត្រឹមត្រូវ នៅ គ្រប់ level។

1

Client Side

Browser / Mobile App Validate ទម្រង់ (form) ជាមុន មុន Submit ដើម្បីធ្វើ UX ឲ្យ Fast ។

2

API Layer

ASP.NET Core Validate Model ដោយ Data Annotations ឬ FluentValidation – ស្រទាប់ចម្បង។

3

Service Layer

Business Logic Validation ដូចជា "Username Already Exists?" ត្រូវធ្វើ ក្នុង Service Layer។

4

Database Layer

Database Constraints (Unique, Not Null, Foreign Key) ជាការ Validate ចុងក្រោយ ពី DB Level។

DATA ANNOTATION

២.៤. Validation ជាមួយ Data Annotation

Data Annotations គឺជា Attributes ដែលអ្នកអភិវឌ្ឍន៍ Apply ទៅ Properties នៅក្នុង Model Class ដើម្បីកំណត់ Validation Rules ។ ជាបច្ចេកទេស Validation ដ៏ងាយបំផុត ហើយ Built-in ជាមួយ ASP.NET Core ។

ងាយស្រួលប្រើ

គ្រាន់តែ Add Attribute ទៅ Property ក្នុង Model Class ហើយ Framework Handle ការ Validation ដោយ Automatic ។

Built-in ASP.NET Core

Data Annotations ត្រូវបាន Include ក្នុង .NET Framework ដោយ Default – មិនចាំបាច់ Install Package ផ្សេង។

Declarative Style

Validation Rules ត្រូវបាន Declare ដោយ Attributes ដែលងាយអានហើយ Logic ច្បាស់លាស់ ។

ការពន្យល់

ក. តើ Data Annotation ជាអ្វី?

Data Annotations គឺជា Attributes ដែលស្ថិតនៅក្នុង **System.ComponentModel.DataAnnotations** Namespace ក្នុង .NET ។ ពួកវាអនុញ្ញាតឲ្យអ្នកអភិវឌ្ឍន៍ Add Metadata ទៅ Model Classes ដើម្បីបញ្ជាក់ Validation Rules, Display Names, Data Types ជាដើម ។

Attribute-based Approach

Data Annotations ប្រើ C# Attributes (decorator pattern) ដើម្បីបញ្ជាក់ Validation Rules ដែលអ្នកគ្រាន់តែ Apply ទៅ Property ដូចជា **[Required]**, **[MaxLength(100)]** ជាដើម ។

Metadata Storage

Data Annotations ផ្ទុក Metadata (Information about Data) ជាមួយ Model Class ខ្លួនឯង ដែលធ្វើឲ្យ Validation Rules ច្បាស់លាស់ ហើយ Maintainable ។

Multiple Purposes

Data Annotations ប្រើបានក្នុង ASP.NET Core MVC Validation, Entity Framework Core (Database Schema), ហើយក៏ប្រើជាមួយ Swagger Documentation ផងដែរ ។

ពេលណាប្រើ

ខ. ពេលណាត្រូវប្រើ Data Annotation

Data Annotations ល្អបំផុតសម្រាប់ Validation Rules ដែល Simple ហើយ Straight-forward ។ ពួកវា Ideal ក្នុង Scenarios ដូចខាងក្រោម ប៉ុន្តែ មិនសាកសម ក្នុង Scenarios ដែល Complex ។

✅ ល្អ ប្រើ Data Annotation

- Simple validation rules (Required, Length, Range)
- DTO / Request models ដែល Validation មិន Complex
- Project តូច ឬ Simple CRUD API
- Validation ដែលមិនពឹងផ្អែក Database ឬ External Service
- ចង់ Quick Setup ដោយមិន Install Extra Package

❌ មិនល្អ ប្រើ Data Annotation

- Validation Rules ដែល Complex, Multi-step
- Conditional Validation (Validate A បើ B ជាក់លាក់)
- Database-dependent Validation (Check Unique)
- Cross-field Validation (Field A ត្រូវ Less Than Field B)
- Unit Testing ចង់ Test Validation Logic ដោយ Isolated

ឯកសារយោង

គ. Data Annotation ដែលប្រើញឹកញាប់

ខាងក្រោមនេះជា Data Annotation Attributes ដែលអ្នកអភិវឌ្ឍន៍ ASP.NET Core ប្រើញឹកញាប់បំផុត ពួកវាស្ថិតនៅក្នុង Namespace **System.ComponentModel.DataAnnotations**។

Annotation	ការពណ៌នា	ឧទាហរណ៍
[Required]	Field ត្រូវតែមានតម្លៃ (ហ្មង់ null ឬ empty)	[Required(ErrorMessage = "Name is required")]
[StringLength]	កំណត់ Min/Max Length សម្រាប់ String	[StringLength(100, MinimumLength = 2)]
[MaxLength]	ចំនួន Characters អតិបរមា	[MaxLength(200)]
[MinLength]	ចំនួន Characters អប្បបរមា	[MinLength(6)]
[Range]	ទំហំតម្លៃ Min ទៅ Max	[Range(1, 100)]
[EmailAddress]	ពិនិត្យ Email Format	[EmailAddress]
[Phone]	ពិនិត្យ Phone Number Format	[Phone]
[Url]	ពិនិត្យ URL Format	[Url]
[RegularExpression]	ពិនិត្យ Custom Pattern	[RegularExpression(@"^[A-Za-z]+$")]
[Compare]	ប្រៀបធៀប ២ Fields	[Compare("Password")]

អនុវត្ត

ឃ. អនុវត្តជាមួយ Data Annotation

ខាងក្រោមនេះជាឧទាហរណ៍ជាក់ស្តែងនៃការ Apply Data Annotations ទៅ DTO Model Class ដែលប្រើសម្រាប់ User Registration API Endpoint ។

```
public class RegisterUserDto
{
    [Required(ErrorMessage = "Username is required.")]
    [StringLength(50, MinimumLength = 3,
        ErrorMessage = "Username must be 3-50 characters.")]
    public string Username { get; set; }

    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Invalid email format.")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Password is required.")]
    [MinLength(8, ErrorMessage = "Password must be at least 8 chars.")]
    [RegularExpression(@"^(?=.*[A-Z])(?=.*[0-9]).+$",
        ErrorMessage = "Password needs uppercase and number.")]
    public string Password { get; set; }

    [Required(ErrorMessage = "Confirm password is required.")]
    [Compare("Password",
        ErrorMessage = "Passwords do not match.")]
    public string ConfirmPassword { get; set; }

    [Range(18, 100,
        ErrorMessage = "Age must be between 18 and 100.")]
    public int Age { get; set; }
}
```

នៅក្នុង Controller ប្រសិនបើ **[ApiController]** Attribute ត្រូវបាន Apply ការ Validate ModelState.IsValid ត្រូវបានធ្វើ ដោយ Automatic – ប្រសិនបើ Validation Fail, Framework ត្រឡប់ **400 Bad Request** Response ដោយ Automatic ហើយ Action Method Code នឹង មិនដំណើរការ ។

✓ FLUENT VALIDATION

៣. អំពី Fluent Validation

FluentValidation គឺជា Open-source Library សម្រាប់ .NET ដែលផ្តល់នូវវិធីបញ្ចេញ Validation Rules ក្នុង Code ដោយ Fluent Interface (Method Chaining) ជំនួសឲ្យ Data Annotations ។ Library នេះ បង្កើតដោយ **Jeremy Skinner** ហើយ ជា Library ដ៏ពេញនិយមបំផុត ក្នុង .NET Ecosystem សម្រាប់ Complex Validation ។

Fluent API

ប្រើ Method Chaining ដើម្បី Define Validation Rules ក្នុង Code ដែលអានបានងាយ ហើយ Expressive ។

Powerful Rules

Support Conditional Validation, Cross-field Validation, Async Validation, Custom Validators ដែល Data Annotations មិនអាចធ្វើបាន ។

Testable

Validators ជា Separate Classes ដែល Unit Test បានងាយ ដោយ Isolated ពី Controller ។

ការពន្យល់

៣.១. អ្វីជា Fluent Validation?

FluentValidation ប្រើ **AbstractValidator<T>** Base Class ដើម្បីបង្កើត Validator Classes ។ ក្នុង Constructor ផ្ទាល់ខ្លួន អ្នកអភិវឌ្ឍន៍ Define Rules ដោយ Method Chaining ពី **RuleFor()** Method ។ ន Validator Class ត្រូវបាន Registered ក្នុង DI Container ហើយ ASP.NET Core ប្រើ Validator ដោយ Automatic ។

```
// ឧទាហរណ៍ Fluent Validator ងាយ
public class CreateProductValidator
 : AbstractValidator<CreateProductDto>
{
 public CreateProductValidator()
 {
 RuleFor(x => x.Name)
 .NotEmpty()
 .WithMessage("Product name is required.")
 .MaximumLength(100)
 .WithMessage("Name cannot exceed 100 characters.");

 RuleFor(x => x.Price)
 .GreaterThan(0)
 .WithMessage("Price must be greater than 0.");
 }
}
```

ហេតុផល

៣.២. ហេតុអ្វីត្រូវប្រើ Fluent Validation?

1 Separation of Concerns

Validation Logic ស្ថិតក្នុង Separate Validator Class ជំនួសឲ្យ ស្ថិត ក្នុង Model Class ។ នេះ ធ្វើឲ្យ Single Responsibility Principle ត្រូវបាន Follow ហើយ Code ងាយ Maintain ។

2 Complex Validation Made Easy

Fluent Validation Support Conditional Validation (.When()), Cross-field Validation, Async Validation (Database Check) ហើយ Custom Validators ដែល Data Annotations មិនអាច Handle ។

3 Excellent Unit Testability

Validators ជា Classes ធម្មតា ដែល Unit Test បានងាយដោយ Isolated ។ FluentValidation Package ផ្តាល់ផ្តល់ Test Extensions ដូចជា **ShouldHaveValidationErrorFor()** ។

4 Reusable Validators

អ្នកអភិវឌ្ឍន៍ អាច Reuse Validators, Compose Validators ធំ ពី Validators តូចៗ ហើយ Share Common Rules ក្នុង Application ។

ការតំឡើង

៣.៣. តំឡើង Fluent Validation

ដើម្បី Install FluentValidation ក្នុង ASP.NET Core Project អ្នកត្រូវ Install NuGet Package ២ ។ Package ទី ១ ជា Core Library ហើយ Package ទី ២ ជា Integration Package សម្រាប់ ASP.NET Core ។

```
# Install via .NET CLI
dotnet add package FluentValidation
dotnet add package FluentValidation.AspNetCore

# ឬ Install via NuGet Package Manager
Install-Package FluentValidation
Install-Package FluentValidation.AspNetCore
```

ក្រោយ Install Package ត្រូវ Register FluentValidation ក្នុង **Program.cs** ។

```
// Program.cs
using FluentValidation;
using FluentValidation.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Register FluentValidation
builder.Services.AddFluentValidationAutoValidation();
builder.Services.AddValidatorsFromAssemblyContaining<Program>();
```

Method **AddValidatorsFromAssemblyContaining<Program>()** ធ្វើ Scan ទូទាំង Assembly ហើយ Register Validators ទាំងអស់ ក្នុង DI Container ដោយ Automatic ។ ដូច្នេះ មិនចាំបាច់ Register Validators ម្តងម្នាក់ ។

មុខងារ

៣.៤. មុខងារ Fluent Validation សំខាន់

Method	ការពណ៌នា	ឧទាហរណ៍
NotEmpty()	Field ត្រូវតែមិន Null, Empty, Whitespace	RuleFor(x => x.Name).NotEmpty()
NotNull()	Field ត្រូវតែមិន Null	RuleFor(x => x.Id).NotNull()
MaximumLength()	String Length Max	.MaximumLength(100)
MinimumLength()	String Length Min	.MinimumLength(6)
EmailAddress()	ពិនិត្យ Email Format	.EmailAddress()
GreaterThan()	Number ធំជាង Value	.GreaterThan(0)
LessThan()	Number តូចជាង Value	.LessThan(1000)
InclusiveBetween()	Number ស្ថិតក្នុង Range	.InclusiveBetween(1, 100)
Matches()	Regex Pattern Match	.Matches(@"^[A-Z]")
Must()	Custom Validation Logic	.Must(x => IsUnique(x))
When()	Conditional Validation	.When(x => x.IsActive)
MustAsync()	Async Custom Validation	.MustAsync(CheckDbAsync)
WithMessage()	Custom Error Message	.WithMessage("Name required")
SetValidator()	Nested Validator	.SetValidator(new AddressValidator())

RESPONSE

៣.៥. Validation Error Response

នៅពេល Validation Fail FluentValidation ត្រឡប់ **400 Bad Request** Response ដោយ Default ។ Response Body ស្ថិតក្នុង ProblemDetails Format ដែលមាន **errors** Property ដែល Contains Validation Errors ។

```
// Response Body 400 Bad Request (ProblemDetails format)
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": [
      "Product name is required.",
      "Name cannot exceed 100 characters."
    ],
    "Price": [
      "Price must be greater than 0."
    ]
  },
  "traceId": "00-abc123..."
}
```

Response Format ខាងលើ គឺ Consistent ជាមួយ ProblemDetails Standard ហើយ Client (Frontend, Mobile) អាច Parse Errors ហើយ Display Messages ទៅ User ងាយស្រួល ។ Field-level Errors ជួយ User ដឹងពិតប្រាកដ ថាតើ Field ណាខ្លះ Input ខុស ។

អនុវត្ត – ទៀប

អនុវត្តជាមួយ Fluent Validation (កំរិតទៀប)

ឧទាហរណ៍ Basic ដំបូង – Validate Request DTO ធម្មតា ដោយ Fluent Validation ។ Level នេះ គ្រប់ Basic Rules ដូច Data Annotations ប៉ុន្តែ ក្នុង Fluent Style ។

```
// DTO
public class CreateProductDto
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string Category { get; set; }
}

// Validator (Basic)
public class CreateProductValidator
    : AbstractValidator<CreateProductDto>
{
    public CreateProductValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Name is required.")
            .MaximumLength(100)
            .WithMessage("Name max 100 characters.");

        RuleFor(x => x.Price)
            .GreaterThan(0)
            .WithMessage("Price must be > 0.");

        RuleFor(x => x.Category)
            .NotEmpty()
            .WithMessage("Category is required.");
    }
}
```

ចំណាំ៖ ក្រោយ Register **AddFluentValidationAutoValidation()** Validator ត្រូវបាន Apply ដោយ Automatic – Controller Action Method Code នឹង Run លុះត្រាតែ Validation Pass ។

</> អនុវត្ត – មធ្យម

អនុវត្តជាមួយ Fluent Validation (កំរិតមធ្យម)

Level មធ្យម – Conditional Validation, Cross-field Validation, ហើយ Custom Must() Validator ។

```
public class RegisterUserDto
{
 public string Username { get; set; }
 public string Email { get; set; }
 public string Password { get; set; }
 public string ConfirmPassword { get; set; }
 public bool IsCompany { get; set; }
 public string CompanyName { get; set; }
}

public class RegisterUserValidator
 : AbstractValidator<RegisterUserDto>
{
 public RegisterUserValidator()
 {
 RuleFor(x => x.Username)
 .NotEmpty()
 .MinimumLength(3).MaximumLength(50);

 RuleFor(x => x.Email)
 .NotEmpty().EmailAddress();

 RuleFor(x => x.Password)
 .NotEmpty().MinimumLength(8)
 .Matches(@"[A-Z]")
 .WithMessage("Password needs uppercase.")
 .Matches(@"[0-9]")
 .WithMessage("Password needs a number.");

 // Cross-field Validation
 RuleFor(x => x.ConfirmPassword)
 .Equal(x => x.Password)
 .WithMessage("Passwords must match.");

 // Conditional Validation
 RuleFor(x => x.CompanyName)
 .NotEmpty()
 .WithMessage("Company name required for companies.")
 .When(x => x.IsCompany);
 }
}
```

អនុវត្ត – ខ្ពស់

អនុវត្តជាមួយ Fluent Validation (កំរិតខ្ពស់)

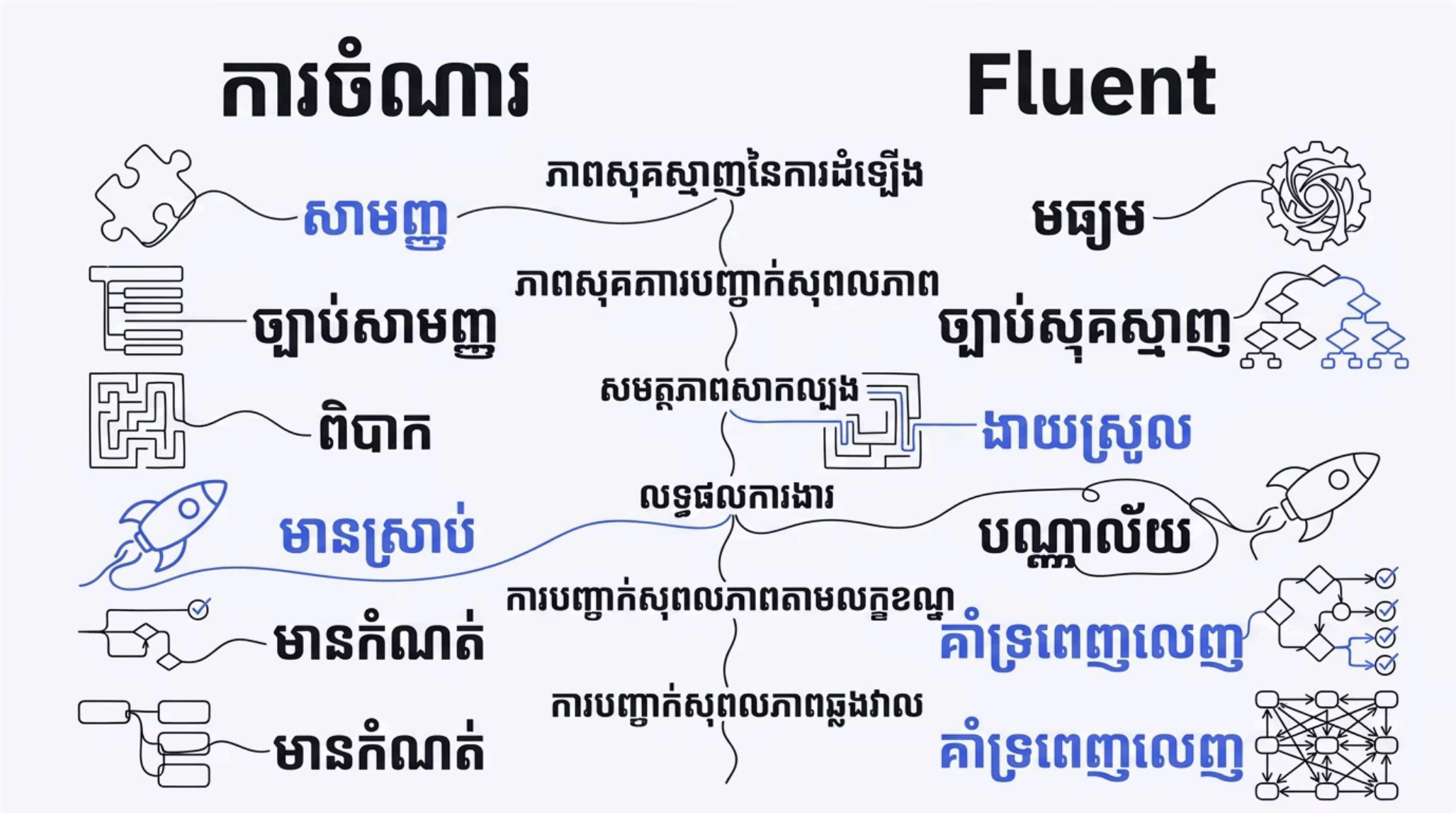
Level ខ្ពស់ – Async Database Validation, Nested Object Validation, ហើយ Custom Reusable Validator Extensions ។

```
public class CreateOrderValidator
 : AbstractValidator<CreateOrderDto>
{
 private readonly IUserRepository _userRepo;
 private readonly IProductRepository _productRepo;

 public CreateOrderValidator(
 IUserRepository userRepo,
 IProductRepository productRepo)
 {
 _userRepo = userRepo;
 _productRepo = productRepo;

 // Async DB Validation
 RuleFor(x => x.UserId)
 .NotEmpty()
 .MustAsync(UserExistsAsync)
 .WithMessage("User not found.");

 RuleFor(x => x.ProductId)
 .NotEmpty()
 .MustAsync(ProductExistsAsync)
 .WithMessage("Product not found.");

 // Nested Object Validation
 RuleFor(x => x.ShippingAddress)
 .NotNull()
 .SetValidator(new AddressValidator());

 RuleFor(x => x.Quantity)
 .GreaterThan(0)
 .LessThanOrEqualTo(100);
 }

 private async Task<bool> UserExistsAsync(
 int userId, CancellationToken ct)
 => await _userRepo.ExistsAsync(userId);

 private async Task<bool> ProductExistsAsync(
 int productId, CancellationToken ct)
 => await _productRepo.ExistsAsync(productId);
}
```

ការប្រៀបធៀប

២.៥. Data Annotation vs Fluent Validation

ការជ្រើសរើស Validation Approach ត្រឹមត្រូវ គឺ អាស្រ័យ Project ទំហំ និង Complexity ។ ខាងក្រោមជា Comparison ច្បាស់ ដើម្បីជួយ Decision ។



សម្រាប់ Project ធំ ដែលត្រូវការ Complex Business Logic Validation, Database Checks, ឬ Unit Testing ចំពោះ Validation Logic គួរជ្រើស **FluentValidation** ។ សម្រាប់ Project តូច ឬ Simple CRUD API ដែល Validation Rules មិន Complex **Data Annotations** ល្អព្រោះ Setup រហ័ស ហើយ No Extra Library ។

</> អនុវត្ត – មធ្យម

អនុវត្តជាមួយ ProblemDetails (កំរិតមធ្យម)

ឧទាហរណ៍ Level មធ្យម – Custom Exception Middleware ដែល Map Custom Exceptions ទៅ ProblemDetails Response ។

```
// Custom Exceptions
public class NotFoundException : Exception
{
 public NotFoundException(string message) : base(message) { }
}
public class ValidationException : Exception
{
 public IDictionary<string, string[]> Errors { get; }
 public ValidationException(
 IDictionary<string, string[]> errors)
 : base("Validation failed.")
 {
 Errors = errors;
 }
}

// Custom Exception Handler (IExceptionHandler - .NET 8+)
public class GlobalExceptionHandler : IExceptionHandler
{
 private readonly ILogger<GlobalExceptionHandler> _logger;
 public GlobalExceptionHandler(
 ILogger<GlobalExceptionHandler> logger)
 {
 _logger = logger;
 }

public async ValueTask<bool> TryHandleAsync(
HttpContext context,
Exception exception,
CancellationToken ct)
{
_logger.LogError(exception, "Unhandled exception.");

var (statusCode, title) = exception switch
{
NotFoundException =>
(StatusCodes.Status404NotFound, "Not Found"),
ValidationException =>
(StatusCodes.Status400BadRequest, "Validation Error"),
_ =>
(StatusCodes.Status500InternalServerError,
"Internal Server Error")
 };

 var problem = new ProblemDetails
 {
 Status = statusCode,
 Title = title,
 Detail = exception.Message
 };

 context.Response.StatusCode = statusCode;
 await context.Response.WriteAsJsonAsync(problem, ct);
 return true;
 }
}
```

អនុវត្ត – ខ្ពស់

អនុវត្តជាមួយ ProblemDetails (កំរិតខ្ពស់)

Level ខ្ពស់ – Custom ProblemDetails Factory ដែល Add traceId, CorrelationId, ហើយ Validation Errors ក្នុង Standard ProblemDetails Format ។

```
// Custom ProblemDetails Factory
public class CustomProblemDetailsFactory
 : ProblemDetailsFactory
{
 private readonly ApiBehaviorOptions _options;

 public CustomProblemDetailsFactory(
 IOptions<ApiBehaviorOptions> options)
 {
_options = options.Value;
}

public override ProblemDetails CreateProblemDetails(
HttpContext httpContext,
int? statusCode = null,
string? title = null, ...)
{
var problem = new ProblemDetails
{
Status = statusCode ?? 500,
Title = title,
Detail = detail,
Instance = instance
};

// Add TraceId for correlation/debugging
problem.Extensions["traceId"] =
Activity.Current?.Id ??
httpContext.TraceIdentifier;

return problem;
}

public override ValidationProblemDetails
CreateValidationProblemDetails(
HttpContext httpContext,
ModelStateDictionary modelState, ...)
{
var problem = new ValidationProblemDetails(modelState)
{
Status = statusCode ?? 400
 };

 // Add TraceId
 problem.Extensions["traceId"] =
 Activity.Current?.Id ??
 httpContext.TraceIdentifier;

 return problem;
 }
}

// Register in Program.cs
builder.Services.AddSingleton<ProblemDetailsFactory,
 CustomProblemDetailsFactory>();
```

HEALTH CHECK

៤. អំពី Health Check ក្នុង ASP.NET Core Web API

Health Checks គឺជាមេកានិចសម្រាប់ត្រួតពិនិត្យថា Application កំពុងដំណើរការល្អ ឬ មានបញ្ហាអ្វីមួយ ដូចជា Database down, External API fail, Memory issue ជាដើម ។ Health Check Endpoint ត្រឡប់ Status ដែល Orchestration System (Kubernetes, Docker) ប្រើ ដើម្បីសម្រេចថា ត្រូវ Route Traffic ឬ Restart Container ។

Service Monitoring

ត្រួតពិនិត្យ API, Database, Cache, External Services ថាដំណើរការ ឬ Down ។

Orchestration Integration

Kubernetes, Docker Swarm, Load Balancers ប្រើ Health Checks ដើម្បីសម្រេចថា Container Healthy ហើយ Route Traffic ។

Prevent Bad Traffic

ការ Stop Traffic ទៅ Instance ដែល Unhealthy ការពារ User ពីការ Hit API ដែលមានបញ្ហា ។

ហេតុផល

៤.១. ហេតុអ្វីត្រូវការ Health Checks?

ក្នុង Production Environment ពិសេស Microservices Architecture Health Checks ជា Critical Component ។ ពួកវាជួយ Operations Team, Monitoring Tools, ហើយ Orchestration Systems ដឹងពី State ពិត នៃ Application ។

ត្រួតពិនិត្យ API Run ឬ អត់

ដឹង ភ្លាម ថា API Process ដំណើរការ ហើយ Accepting Requests ។ Liveness Probe ធ្វើការ Check នេះ ហើយ Kubernetes Restart Container ប្រសិនបើ Liveness Fail ។

ត្រួតពិនិត្យ Database Connection

ប្រសិន Database Connection Fail, Application Respond Requests ជារឿយៗ Error ។ Health Check ដឹង ភ្លាម ហើយ Prevent Traffic ទៅ Instance ខូច ។

ត្រួតពិនិត្យ External Services

Check Connectivity ទៅ External APIs, Redis Cache, Message Queues ។ Degraded Status ជួយ Report ថា Service ដំណើរការ Partially ។

ប្រើជាមួយ Kubernetes / Load Balancer

Kubernetes ប្រើ Liveness Probe ដើម្បី Restart Container Dead ហើយ Readiness Probe ដើម្បី Remove Instance ពី Load Balancer ។

ប្រភេទ

៤.២. ប្រភេទនៃ Health Checks

Liveness Check

ត្រួតពិនិត្យថា **Application Process ដំណើរការ ហើយ Responding ។** ប្រសិនបើ Liveness Fail Kubernetes Restart Container ។ Check ធម្មតា៖ API Process Alive, No Deadlock, Memory Usage Acceptable ។ Endpoint: **/health/live** ។

Readiness Check

ត្រួតពិនិត្យថា **Application Ready ទទួល Traffic** ។ ប្រសិនបើ Readiness Fail Load Balancer ឈប់ Route Traffic ទៅ Instance នេះ ។ Check ធម្មតា: Database Connection, Cache Connected, Required Services Available ។ Endpoint: **/health/ready** ។

Custom Health Checks

Health Checks ផ្ទាល់ខ្លួនដែល Check ដំណើរការ ជាក់លាក់ ។ ឧទាហរណ៍ Check External Payment API, Check Disk Space, Check Message Queue Depth ។ Custom Checks Implement Interface **IHealthCheck** ។

STATUS TYPES

៤.៣. Health Status Types

ASP.NET Core Health Checks ត្រឡប់ Status មួយក្នុងចំណោម ៣ ប្រភេទ ។ Status នេះ ត្រូវបានប្រើដោយ Orchestration Systems ហើយ Monitoring Tools ។

✅ Healthy

Component ទាំងអស់ដំណើរការធម្មតា ។ Application Ready ទទួល Traffic ។ HTTP Status Code: **200 OK** ។

⚠️ Degraded

Component មួយ ឬ ច្រើន ដំណើរការ Partial ។ Application ដំណើរការ ប៉ុន្តែ Performance ធ្លាក់ ។ HTTP Status Code: **200 OK** (ប៉ុន្តែ Body បង្ហាញ Degraded) ។

❌ Unhealthy

Component សំខាន់ Fail ។ Application មិន Ready ទទួល Traffic ។ HTTP Status Code: **503 Service Unavailable** ។

IMPLEMENT

៤.៤. របៀប Implement ក្នុង ASP.NET Core

ASP.NET Core ផ្តល់ Built-in Health Check Framework ។ ការ Implement ងាយស្រួល – Register Services ហើយ Map Endpoints ។

ការ Config Health Check Endpoints ឲ្យ Separate (ឧ. **/health/live** vs **/health/ready**) ជួយ Kubernetes ដឹងពី Liveness ហើយ Readiness ដោយ Separate ។ ការ Config **HealthCheckOptions** អ្នក អាច Customize Response Format ឲ្យ Detail ជាង Default ដែលបង្ហាញ "Healthy" ជា Plain Text ។

BASIC HEALTH CHECK

ก. Basic Health Check

การ Setup Basic Health Check ใน ASP.NET Core ง่าย ๆ ASP.NET Core ให้ Built-in Health Check Package มาให้

```
// Program.cs - Basic Setup
var builder = WebApplication.CreateBuilder(args);

// 1. Register Health Check Services
builder.Services.AddHealthChecks()
 .AddCheck("self", () => HealthCheckResult.Healthy())
 .AddCheck<CustomHealthCheck>("custom_check");

var app = builder.Build();

// 2. Map Health Check Endpoints
app.MapHealthChecks("/health/live");
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
 ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});

// Custom Health Check
public class CustomHealthCheck : IHealthCheck
{
 public Task<HealthCheckResult> CheckHealthAsync(
 HealthCheckContext context,
 CancellationToken cancellationToken = default)
 {
 // Custom check logic
 bool isHealthy = CheckServiceStatus();

 return Task.FromResult(isHealthy
 ? HealthCheckResult.Healthy("Service OK")
 : HealthCheckResult.Unhealthy("Service down"));
 }
}
```

DATABASE HEALTH CHECK

ខ. Database Health Check

Database Health Check ពិនិត្យ Database Connection ។ ASP.NET Core ផ្តល់ Built-in Health Check Extensions សម្រាប់ SQL Server, PostgreSQL, MySQL ជាដើម ។

```
// Install Package ជាមុន
// dotnet add package AspNetCore.HealthChecks.SqlServer
// dotnet add package AspNetCore.HealthChecks.NpgSql
// dotnet add package AspNetCore.HealthChecks.Redis

// Program.cs
builder.Services.AddHealthChecks()
 // SQL Server Health Check
.AddSqlServer(
connectionString: builder.Configuration
.GetConnectionString("DefaultConnection"),
name: "database",
failureStatus: HealthStatus.Unhealthy,
tags: new[] { "db", "sql" })

// Redis Health Check
.AddRedis(
redisConnectionString: builder.Configuration
.GetConnectionString("Redis"),
name: "redis",
failureStatus: HealthStatus.Degraded,
tags: new[] { "cache", "redis" })

// Custom DB Check with EF Core
.AddDbContextCheck<AppDbContext>(
name: "ef_core_db",
 failureStatus: HealthStatus.Unhealthy);

// Map with Tags (Liveness vs Readiness)
app.MapHealthChecks("/health/live", new HealthCheckOptions
{
 Predicate = check => check.Tags.Contains("live")
});

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
 Predicate = check =>
 check.Tags.Contains("db") ||
 check.Tags.Contains("cache")
});
```

MONITORING

៦. អំពី Monitoring and Observability

Monitoring and Observability គឺជា Practices ដែលជួយ Team ដឹងពី State ពិត, Health, ហើយ Performance របស់ Application ក្នុង Production ។ ក្នុងពិភព Distributed Systems (Microservices) Observability ធ្លាក់ជា Critical Need ព្រោះ Debugging ក្នុង Production Complex ណាស់ ។

Proactive vs Reactive

Monitoring ជា Reactive (Alert ពេល Problem) ។ Observability ជា Proactive – ដឹងពី Root Cause ដោយ Data ។

Production Insights

ដឹងពី Performance Bottlenecks, Error Rates, User Behavior, Resource Usage ក្នុង Real-time ។

Faster MTTR

Mean Time to Recover (MTTR) ធ្លាក់ ព្រោះ Data ពី Logs, Metrics, Traces ជួយ Identify Root Cause ក្នុង Minutes ជំនួស Hours ។

OBSERVABILITY

៦.១. What is Observability?

ជាទូទៅ **Observability មានសសរចម្បង ៣ (Three Pillars)** ដែលជាមូលដ្ឋានគ្រឹះ ។ សសរទាំង ៣ ជួយ Provide Full Picture ពី System State ។

Logs – ដឹងថា "អ្វីកើតឡើង"

Logs គឺជា Timestamped Records នៃ Events ដែលកើតឡើងក្នុង System ។ ពួកវាជួយ Answer: "Request ណាខ្លះ Failed?", "Error Message ជាអ្វី?", "User X ធ្វើអ្វី?" ។ Tools: Serilog, NLog, Microsoft.Extensions.Logging ។

Metrics – វាស់ "ប្រព័ន្ធដំណើរការល្អប៉ុណ្ណា"

Metrics គឺជា Numeric Measurements ដែល Aggregate ក្នុង Time ។ ពួកវា Answer: "Request Count ប៉ុន្មាន?", "Average Response Time ជាអ្វី?", "Error Rate ជា %" ។ Tools: Prometheus, OpenTelemetry Metrics ។

Traces – ដឹង "បញ្ហានៅទីណាក្នុង Request Flow"

Distributed Traces Track ការ Journey ពេញលេញ នៃ Request ក្នុង System ។ ពួកវា Answer: "Request ចំណាយ ២s នៅ Service ណា?", "Database Query Slow នៅ Endpoint ណា?" ។ Tools: Jaeger, Zipkin, OpenTelemetry Tracing ។

OPENTELEMETRY

៦.២. ប្រើ OpenTelemetry ដើម្បី Collect Observability Data

OpenTelemetry (OTel) គឺជា CNCF Open-source Standard Framework សម្រាប់ Collect, Process, ហើយ Export Telemetry Data (Logs, Metrics, Traces) ។ OTel ជា Vendor-neutral ដូច្នេះ ប្តូរ Backend (Jaeger → Zipkin → DataDog) ដោយ Config Change មិន Code Change ។

```
// dotnet add package OpenTelemetry.Extensions.Hosting
// dotnet add package OpenTelemetry.Instrumentation.AspNetCore
// dotnet add package OpenTelemetry.Instrumentation.Http
// dotnet add package OpenTelemetry.Exporter.Console
// dotnet add package OpenTelemetry.Exporter.Otlp

// Program.cs
builder.Services.AddOpenTelemetry()
    .WithTracing(tracing =>
    {
        tracing
        .SetResourceBuilder(ResourceBuilder
            .CreateDefault()
            .AddService("MyApi"))
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddOtlpExporter(opts =>
        {
            opts.Endpoint = new Uri(
                "http://localhost:4317");
        });
    })
    .WithMetrics(metrics =>
    {
        metrics
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddPrometheusExporter();
    });
```

OpenTelemetry Auto-Instrumentation ធ្វើ Instrument ដោយ Automatic ចំពោះ ASP.NET Core Requests, HTTP Client Calls, EF Core Queries ។ ការ Install Package **AddAspNetCoreInstrumentation()** ជា Start Point ។

PROMETHEUS

៦.៣. ប្រើ Prometheus ដើម្បី Collect និង Query Metrics

Prometheus គឺជា Open-source Monitoring System ហើយ Time-series Database ដែល Pull Metrics ពី Application ។ Prometheus Scrape Metrics Endpoint (ទូទៅ **/metrics**) ហើយ Store ក្នុង Time-series Format ។

```
// ការ Setup Prometheus Endpoint ក្នុង ASP.NET Core
// dotnet add package prometheus-net.AspNetCore

// Program.cs
builder.Services.AddOpenTelemetry()
    .WithMetrics(metrics =>
    {
        metrics
            .AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddRuntimeInstrumentation()
            // Export ទៅ Prometheus format
            .AddPrometheusExporter();
    });

var app = builder.Build();

// Map Prometheus Scrape Endpoint
app.MapPrometheusScrapingEndpoint("/metrics");
```

Prometheus Query Language គឺ **PromQL** ។ ឧទាហរណ៍ Queries ដែលប្រើញឹកញាប់ : **rate(http_requests_total[5m])** (Request Rate), **histogram_quantile(0.99, http_request_duration_seconds_bucket)** (P99 Latency) ។ Prometheus ក៏ Support Alerting Rules ផងដែរ ។

GRAFANA

៦.៤. ប្រើ Grafana ដើម្បី Create Dashboards

Grafana គឺជា Open-source Analytics & Visualization Platform ។ Grafana Connect ទៅ Data Sources ច្រើន (Prometheus, Loki, Jaeger, PostgreSQL) ហើយ Create Interactive Dashboards ។

01

Connect Data Sources

Add Prometheus ជា Data Source ក្នុង Grafana (URL: http://prometheus:9090) ។ Grafana Support Data Sources ជាង ៥០ ។

02

Import Dashboards

Grafana Dashboard Library (grafana.com/dashboards) មាន Community Dashboards Ready ។ Dashboard ID 10427 ជា ASP.NET Core Dashboard ពេញនិយម ។

03

Create Custom Panels

Build Custom Panels ដោយ PromQL Queries ។ Graph, Stat, Gauge, Table, Heatmap Panels ។

04

Setup Alerts

Grafana Alerting ជូន Notification ទៅ Slack, Email, PagerDuty ពេល Metric ហួស Threshold ។

JAEGER

៦.៥. ប្រើ Jaeger ដើម្បី Collect Traces

Jaeger គឺជា Open-source Distributed Tracing System ។ Jaeger ត្រូវបានបង្កើតដោយ Uber ហើយ Donated ទៅ CNCF ។ Jaeger Help Visualize Request Journeys ក្នុង Microservices ហើយ Identify Performance Bottlenecks ។

```
// dotnet add package OpenTelemetry.Exporter.Jaeger

// Program.cs - Export Traces ទៅ Jaeger
builder.Services.AddOpenTelemetry()
    .WithTracing(tracing =>
    {
        tracing
        .SetResourceBuilder(ResourceBuilder
            .CreateDefault()
            .AddService(
                serviceName: "OrderApi",
                serviceVersion: "1.0.0"))
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddEntityFrameworkCoreInstrumentation()
        // Export ទៅ Jaeger
        .AddJaegerExporter(opts =>
        {
            opts.AgentHost = "localhost";
            opts.AgentPort = 6831;
        });
});

// Custom Activity (Manual Tracing)
private static readonly ActivitySource _activity
    = new ActivitySource("OrderApi");

public async Task ProcessOrder(int orderId)
{
    using var span = _activity.StartActivity("ProcessOrder");
    span?.SetTag("order.id", orderId);

    // ... business logic ...
}
```

Jaeger UI (Port 16686) បង្ហាញ Trace Visualization ។ អ្នក អាច Search Traces ដោយ Service Name, Operation, Duration, Tags ។ Flame Graph ជួយ Visualize ថា Time ចំណាយ នៅ Service ណា ។

HTTP LOGGING

៦.៦. ប្រើ HTTP Logging

HTTP Logging Middleware ក្នុង ASP.NET Core ធ្វើ Log HTTP Requests ហើយ Responses ដោយ Automatic ។ វាជា Built-in Feature ក្នុង ASP.NET Core .NET 6+ ដែលសាកសមជាង Custom Logging ។

```
// Program.cs - Enable HTTP Logging
builder.Services.AddHttpLogging(logging =>
{
    logging.LoggingFields = HttpLoggingFields.All;

    // Log Request Details
    logging.RequestHeaders.Add("Authorization");
    logging.RequestHeaders.Add("X-Correlation-Id");

    // Limit Body Size
    logging.RequestBodyLogLimit = 4096;
    logging.ResponseBodyLogLimit = 4096;

    // Redact Sensitive Headers
    logging.MediaTypeOptions.AddText("application/json");
});

var app = builder.Build();

// Enable Middleware
app.UseHttpLogging();
```

✅ HTTP Logging ល្អ

- Debug API Issues ក្នុង Development
- Audit Trail ចំពោះ Sensitive Endpoints
- Monitor Specific Endpoints
- Built-in, No Extra Package

⚠️ ប្រុងប្រយ័ត្ន

- Log Body ក្នុង Production ប្រុងប្រយ័ត្ន – Performance Impact
- Sensitive Data (Password, Token) ក្នុង Body – Risk
- ប្រើ Log Level ឲ្យ Correct (Debug/Information)
- Redact Sensitive Headers ជានិច្ច

ចំណាំ ។ HTTP Logging ក្នុង Production Environment ទាមទារ ការប្រុងប្រយ័ត្ន ។ ការ Log Body ទាំងមូល ប្រហែលជា Expose Sensitive Data ហើយ Affect Performance ។ ជាទូទៅ Log Headers ហើយ Status Codes ប៉ុណ្ណោះ ក្នុង Production ។